Experiment No. 2: Perceptron with Fixed Increment Learning Algorithm (AND Function)

1. Aim/Purpose

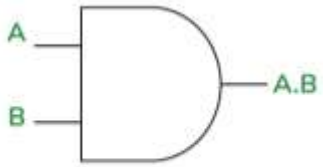
Implement a program for a perceptron with a fixed increment learning algorithm. The program uses the logical AND function as an example. The perceptron is trained until no change in weight is required.

2. Theory

The perceptron is one of the simplest types of artificial neural networks (ANNs) that can be used for binary classification tasks. It consists of:

- Input layer (features of the dataset).
- Weights (adjusted iteratively to minimize classification error).
- Activation function (Binary step function).
- Output layer (predicted class label).

A perceptron learns by adjusting its weights based on misclassified examples using a fixed increment method. The learning process continues until the perceptron classifies all training data correctly or reaches a predefined iteration limit.



Truth Table

A (Input 1)	B (Input 2)	X = (A.B)
0	0	0
0	1	0
1	0	0
1	1	1

Mathematical Model

The perceptron output is given by:

$$y = \begin{cases} 1, & \text{if } \sum_i w_i x_i + b \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

where w_i are the weights, x_i are the input features, and b is the bias.

For an input vector $X = [x_1, x_2]$, weights $W = [w_1, w_2]$, and bias b, the perceptron computes: $y = f(WX + b)$ where f is the activation function defined as: $f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$

3. Algorithm:

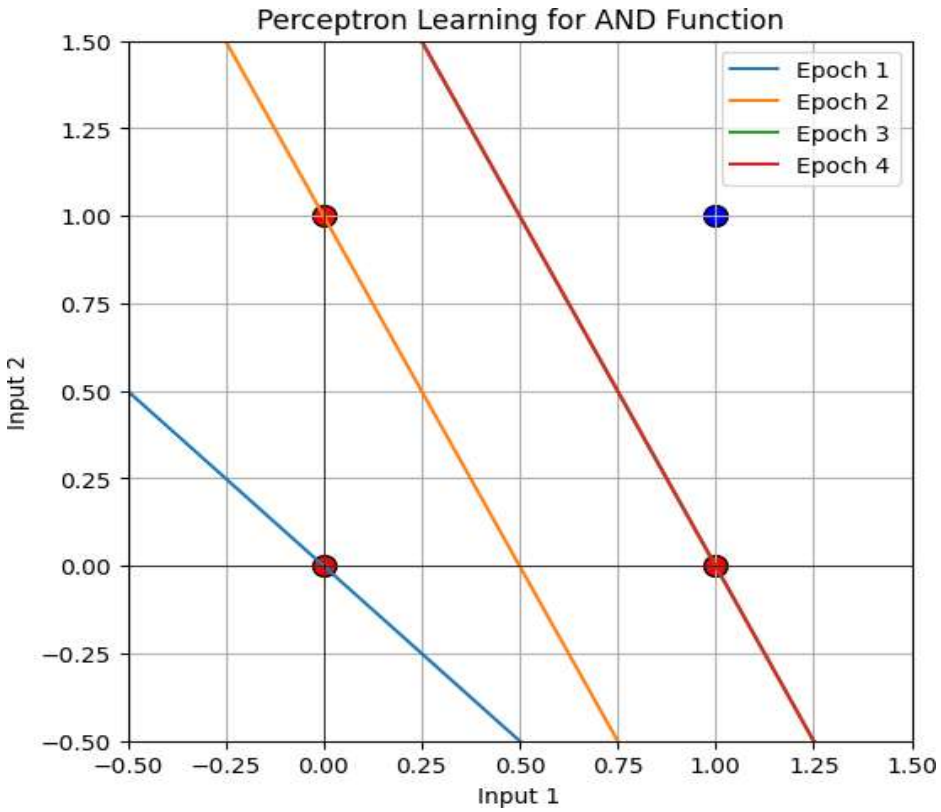
1. Define the AND function as input-output pairs.
2. Initialize weights and bias to small random values.
3. For each training sample:
    - Compute the output using the weighted sum.
    - Update the weights using the rule: where is the learning rate, is the target output, and is the predicted output.
4. Repeat until convergence.
5. Test the trained perceptron on AND function inputs.

4. Implementation:

```
import numpy as np
import matplotlib.pyplot as plt

# AND gate input and output
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])  # Expected output

# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1

# Function to predict output
def predict(inputs):
    summation = np.dot(inputs, weights) + bias
    return 1 if summation >= 0 else 0

# Training the perceptron until convergence
converged = False
iterations = 0
decision_boundaries = []

while not converged:
    iterations += 1
```

```
    previous_weights = weights.copy()
    previous_bias = bias

    for i in range(len(X)):
        prediction = predict(X[i])
        error = y[i] - prediction
        weights += learning_rate * error * X[i]
        bias += learning_rate * error

    # Store decision boundary after each epoch
    if weights[1] != 0:
        x_vals = np.linspace(-0.5, 1.5, 100)
        y_vals = -(weights[0] * x_vals + bias) / weights[1]
        decision_boundaries.append((x_vals, y_vals))

    # Check for convergence
    if np.array_equal(previous_weights, weights) and previous_bias == bias:
        converged = True

# Plot decision boundary evolution
plt.figure(figsize=(6, 6))
for i, (x_vals, y_vals) in enumerate(decision_boundaries):
    plt.plot(x_vals, y_vals, label=f'Epoch {i+1}')

# Plot data points
for i in range(len(X)):
    plt.scatter(X[i][0], X[i][1], c='red' if y[i] == 0 else 'blue',
                marker='o', edgecolors='black', s=100)

plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.title('Perceptron Learning for AND Function')
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid(True)
plt.show()

# Display final weights and number of iterations
print(f'Final Weights: {weights}')
print(f'Final Bias: {bias}')
print(f'Number of Iterations Until Convergence: {iterations}')
```

5. Output




```
Final Weights: [0.2 0.1]

Final Bias: -0.20000000000000004

Number of Iterations Until Convergence: 4
```

6. Conclusion

The perceptron successfully classifies the AND function, demonstrating convergence within a specific number of iterations.